

Fill in the first TODO in the runGDM function to implement gradient descent with momentum

Thought for 2s >

I'll open the notebook, locate the `runGDM` function, and then fill in the first TODO to implement gradient descent with momentum directly in the notebook.

Read `q_sgd_momentum_analysis.ipynb`

I located the `runGDM` function in the notebook. I'll now fill in the first TODO with the standard momentum update for the smoothed gradient and write it back into the notebook cell so you can run it directly.

Edited `q_sgd_momentum_analysis.ipynb`

Thought for 2s >

Here's the momentum update I inserted into `runGDM`:

```
python

def runGDM(maxiter, stepsize, beta):
    w = np.zeros((2, 1))
    grads_m = []
    ws_m = []
    losses_m = []
    for i in range(maxiter):
        grad = 2 * (X.T @ X @ w) - 2 * X.T @ y
        if i == 0:
            smoothed_grad = grad
            #####
            # TODO: implement this method
            #####
            #####
        else:
            smoothed_grad = beta * smoothed_grad + (1 - beta) * grad
            w = w - stepsize * smoothed_grad
            grads_m.append(grad)
            ws_m.append(w)
            loss = np.linalg.norm(y - X @ w) ** 2
            losses_m.append(loss)
    print("Final loss =", loss)
    grads_m = np.array(grads_m).squeeze()
    ws_m = np.array(ws_m).squeeze()
    return grads_m, ws_m, losses_m

maxiter = 100
stepsize = 1e-4
beta = 0.6
grads_m, ws_m, losses_m = runGDM(maxiter, stepsize, beta)
```

Key idea:

- **First step:** `smoothed_grad = grad` (no history yet).
- **Subsequent steps:** use the momentum recursion

`smoothed_grad_t =  $\beta$  * smoothed_grad_{t-1} + (1 -  $\beta$ ) * grad_t.` ✓

You can now run the cell and compare the final loss and plots with plain GD; they should be on a similar scale but show smoother, faster convergence.

Status: First TODO in `runGDM` is now filled with a standard gradient descent with momentum implementation.

Gradient Descent with Momentum

Implement the gradient descent with momentum algorithm. Fill in the missing code for updating the parameters. As a verification step, compare the final loss with the previous part to ensure it is reasonable and not significantly different.

```
def runGDM(maxiter, stepsize, beta):
    w = np.zeros((2, 1))
    grads_m = []
    ws_m = []
    losses_m = []
    for i in range(maxiter):
        grad = 2 * (X.T @ X @ w) - 2 * X.T @ y
        if i == 0:
            smoothed_grad = grad
            #####
            # TODO: implement this method
            #####
        else:
            smoothed_grad = beta * smoothed_grad + (1 - beta) * grad
        w = w - stepsize * smoothed_grad
        grads_m.append(grad)
        ws_m.append(w)
        loss = np.linalg.norm(y - X @ w) ** 2
        losses_m.append(loss)
    print("Final loss =", loss)
    grads_m = np.array(grads_m).squeeze()
    ws_m = np.array(ws_m).squeeze()
    return grads_m, ws_m, losses_m

maxiter = 100
stepsize = 1e-4
beta = 0.6
grads_m, ws_m, losses_m = runGDM(maxiter, stepsize, beta)
```

[5] ✓ 0.0s

Python

... Final loss = 102.7517108728474

→ Correct but different from the solⁿ. The problem has some in-built ambiguity. In-fact makes more sense to use 0.6 (β) with smoothed-grad as a larger value is typically given to the coefficient of momentum by default in typical implementations.

Fill in the second TODO based off the question in the markdown cell above it

Thought for 5s >

I'll fill in the second TODO cell with code that reruns the faster method (momentum) using a larger learning rate, updating `grads_m`, `ws_m`, and `losses_m` so the later comparison plots work correctly.

Edited q_sgd_momentum_analysis.ipynb

Thought for 2s >

Here's what I added for the **second TODO** (cell right after the Question 2 markdown):

```
python

#####
# TODO: implement this method
#####
#####
# From the earlier comparison, gradient descent with momentum converges faster.
# Now we increase its learning rate (stepsize) and rerun it to further
# accelerate convergence, while keeping plain GD as-is for comparison.
# Choose a slightly larger learning rate for momentum (not too large
# to avoid divergence).
stepsize_momentum = 5e-4
# Rerun momentum GD with the larger learning rate
maxiter = 100
beta = 0.6
grads_m, ws_m, losses_m = runGDM(maxiter, stepsize_momentum, beta)
```

How this matches the question above

- Question 2 asks: if one method converges faster, change its learning rate to further accelerate convergence and then re-run that method and compare gradients, parameters, and loss with the other method.
- In earlier code, `runGD` and `runGDM` were already run with:
 - GD: `stepsize = 1e-4`
 - Momentum: `stepsize = 1e-4`
- Here, I:
 - Keep GD fixed (so `grads`, `ws`, `losses` remain as-is).
 - Increase only the momentum learning rate to `5e-4` (still moderate to avoid divergence). ✓
 - Recompute `grads_m`, `ws_m`, `losses_m` using `runGDM` with this larger `stepsize_momentum`.

All the later plotting cells (Compare ... after changing learning rate) reuse `grads`, `ws`, `grads_m`, `ws_m`, `losses`, `losses_m`, so the new run with larger momentum learning rate will be reflected in those comparisons.

Status: Second TODO is now implemented to rerun the faster method (momentum) with a higher learning rate for the requested comparison.

understood question and just increased
to increase the speed of convergence.

Question 2: If one method converges faster, can you try change the learning rate to further accelerate the convergence? Please re-run one method with different learning rate, and compare the gradients, parameters and loss with the other method. Hint: The learning rate cannot be too large, otherwise the function may not converge.

```
#####  
# TODO: implement this method  
#####  
#####  
  
# From the earlier comparison, gradient descent with momentum converges faster.  
# Now we increase its learning rate (stepsize) and rerun it to further  
# accelerate convergence, while keeping plain GD as-is for comparison.  
  
# Choose a slightly larger learning rate for momentum (not too large  
# to avoid divergence).  
stepsize_momentum = 5e-4  
  
# Rerun momentum GD with the larger learning rate  
maxiter = 100  
beta = 0.6  
grads_m, ws_m, losses_m = runGDM(maxiter, stepsize_momentum, beta)
```

[13] ✓ 0.0s

... Final loss = 1.93961309716518e+55

→ correct change.

However, $5e-4$ was later discovered to be too big and I had to reprompt Windows to solve the issue.

The learning rate of $5e-4$ did not work. The loss diverged.

Thought for 3s >

The divergence means the increased learning rate was too aggressive for this problem. I'll lower the momentum learning rate to a more conservative value, update that TODO cell in the notebook, and you can rerun the cells to check that the loss now decreases smoothly.

Edited q_sgd_momentum_analysis.ipynb

Thought for 2s >

You're right: if $5e-4$ caused divergence, it's too large for this quadratic.

I've already updated the TODO cell to use a **smaller but still larger** learning rate:

python

`stepsize_momentum = 2e-4`

and then:

python

```
maxiter = 100
beta = 0.6
grads_m, ws_m, losses_m = runGDM(maxiter, stepsize_momentum, beta)
```

} kept same

What to do now

1. Re-run the notebook cells in order:

- The cell defining `runGDM`.
- The cell with the updated TODO (where `stepsize_momentum = 2e-4`).
- The comparison plotting cells below.

2. Check the behavior:

- `losses_m` should now decrease and not blow up.
- Momentum with $2e-4$ should converge **faster than plain GD at $1e-4$** , especially visible in:
 - Loss plot (`log(loss_i - loss_*)`).
 - Parameter and gradient plots.

If it still looks unstable

Try lowering `stepsize_momentum` slightly, e.g.:

- $1.5e-4$, then $1.2e-4$.

You want the **largest stepsize that still keeps the loss monotonically (or mostly) decreasing** and clearly faster than plain GD.

Summary: I adjusted the second TODO to use a safer larger learning rate ($2e-4$) for momentum, which should avoid divergence while still showing faster convergence for your comparison plots.

Question 2: If one method converges faster, can you try change the learning rate to further accelerate the convergence? Please re-run one method with different learning rate, and compare the gradients, parameters and loss with the other method. *Hint: The learning rate cannot be too large, otherwise the function may not converge.*

```
#####  
# TODO: implement this method  
#####  
  
# From the earlier comparison, gradient descent with momentum converges faster.  
# Now we increase its learning rate (stepsize) and rerun it to further  
# accelerate convergence, while keeping plain GD as-is for comparison.  
  
# Choose a slightly larger learning rate for momentum (not too large  
# to avoid divergence).  
  
stepsize_momentum = 5e-4  
stepsize_momentum = 2e-4  
  
# Rerun momentum GD with the larger learning rate  
maxiter = 100  
beta = 0.6  
grads_m, ws_m, losses_m = runGDM(maxiter, stepsize_momentum, beta)
```

Accept ↩️ Reject ↩️ⓧ

Python

Correct change. The function converges
after this.